AI Product Design: Knowledge Graphs vs Context Graphs

Dr. Vinish Kathuria

INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY
DELHI

Knowledge Graphs

Definition

Structured representation of entities (people, places, concepts) and their relationships.

Context:

Captures what exists and how things connect.

Examples in AI Products:

Recommendation engines linking users → Products → Categories → Reviews

Medical AI linking symptoms → Diseases → Treatments

Knowledge Graphs- Strengths and Limitations

Strengths:

1. Clear, factual relationships.
2. Easy to query and visualize.
3. Supports reasoning and inference.

Limitations:

1. Static: doesn't adapt well to changing situations.
2. May miss contextual meaning (e.g., "bank" as riverbank vs financial institution).

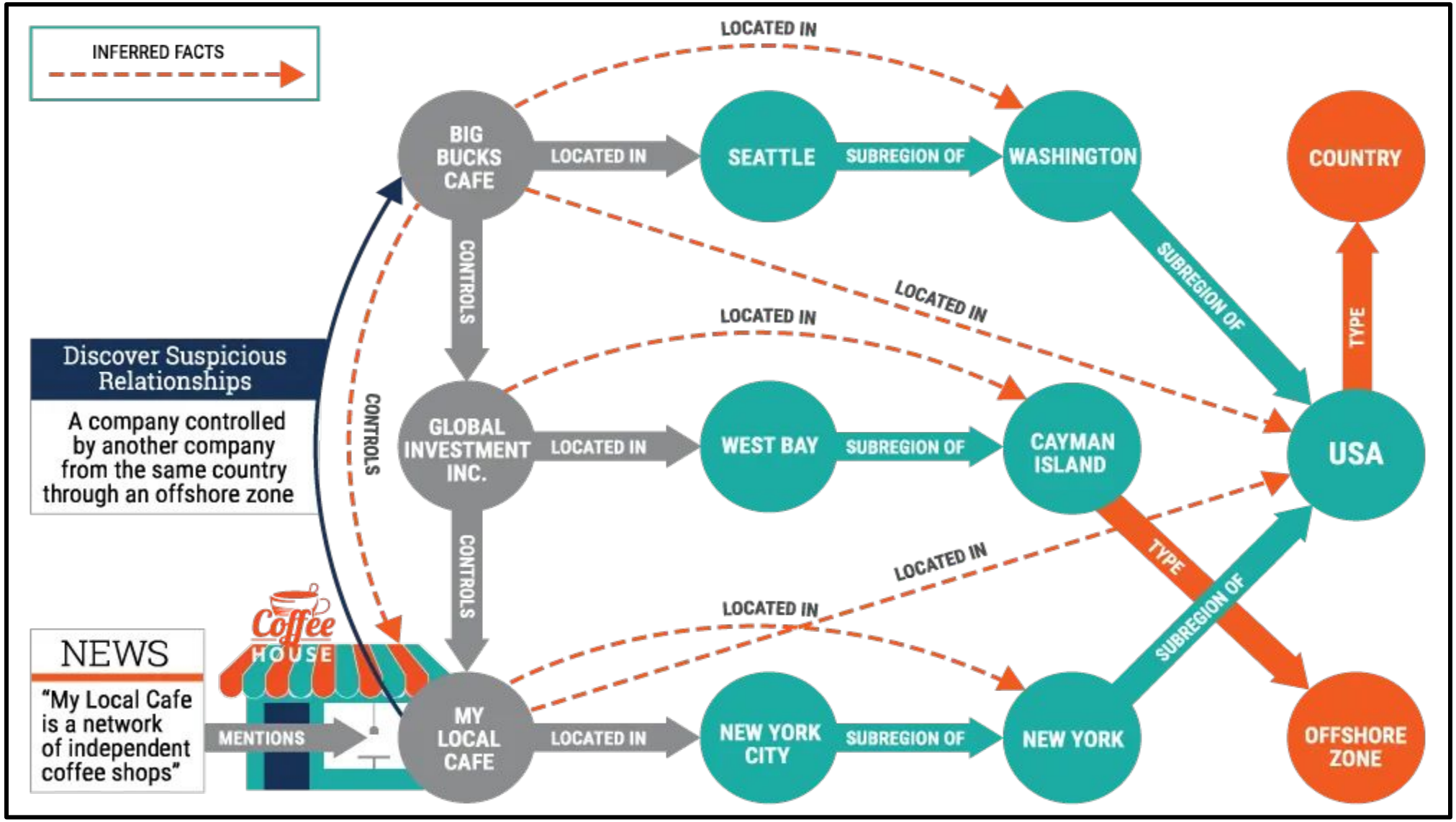

Figure: Knowledge Graph for discovering hidden corporate relationships

https://www.ontotext.com/knowledgehub/fundamentals/what-is-a-knowledge-graph/

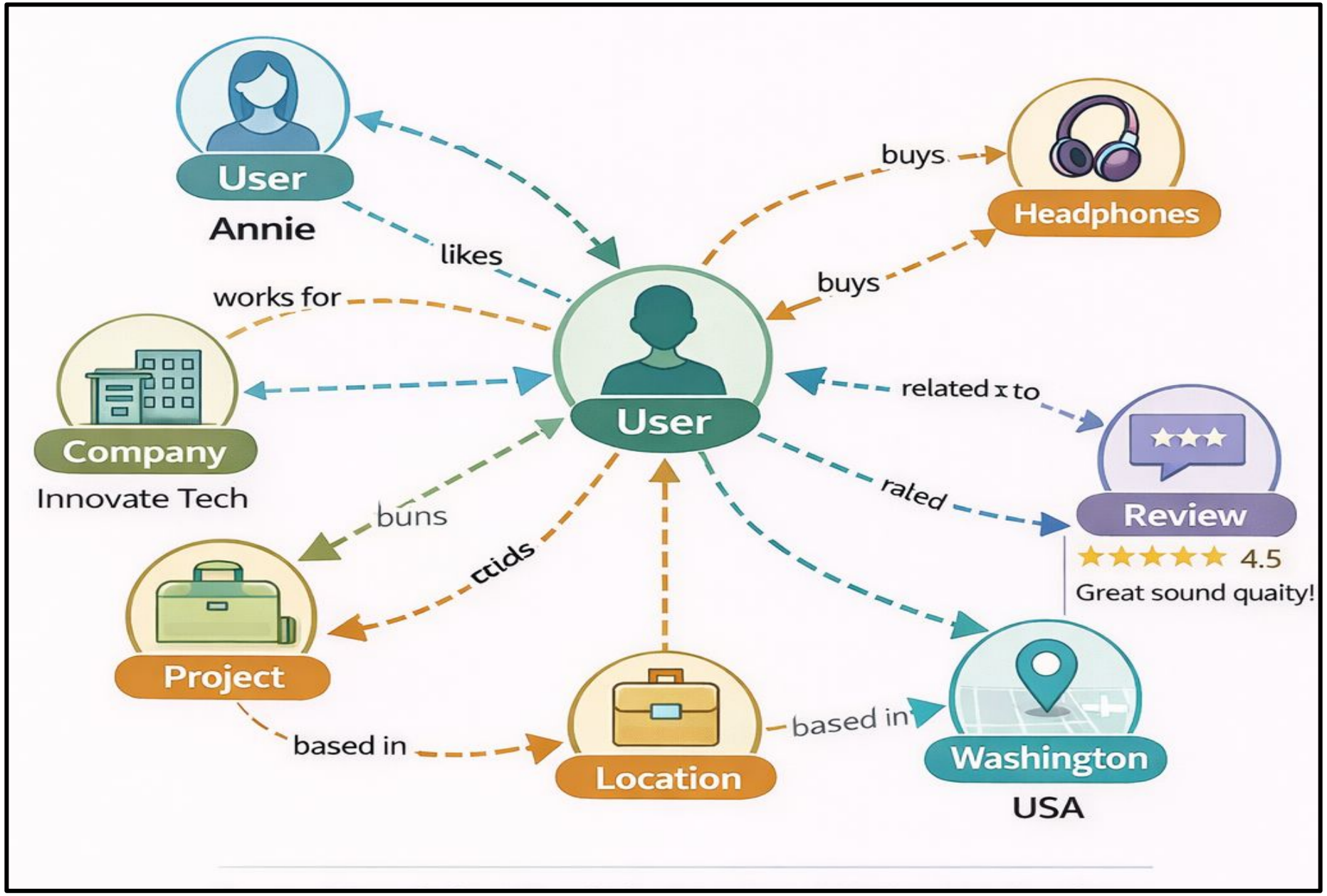

Figure: Knowledge Graph for Structured Entity Relationships

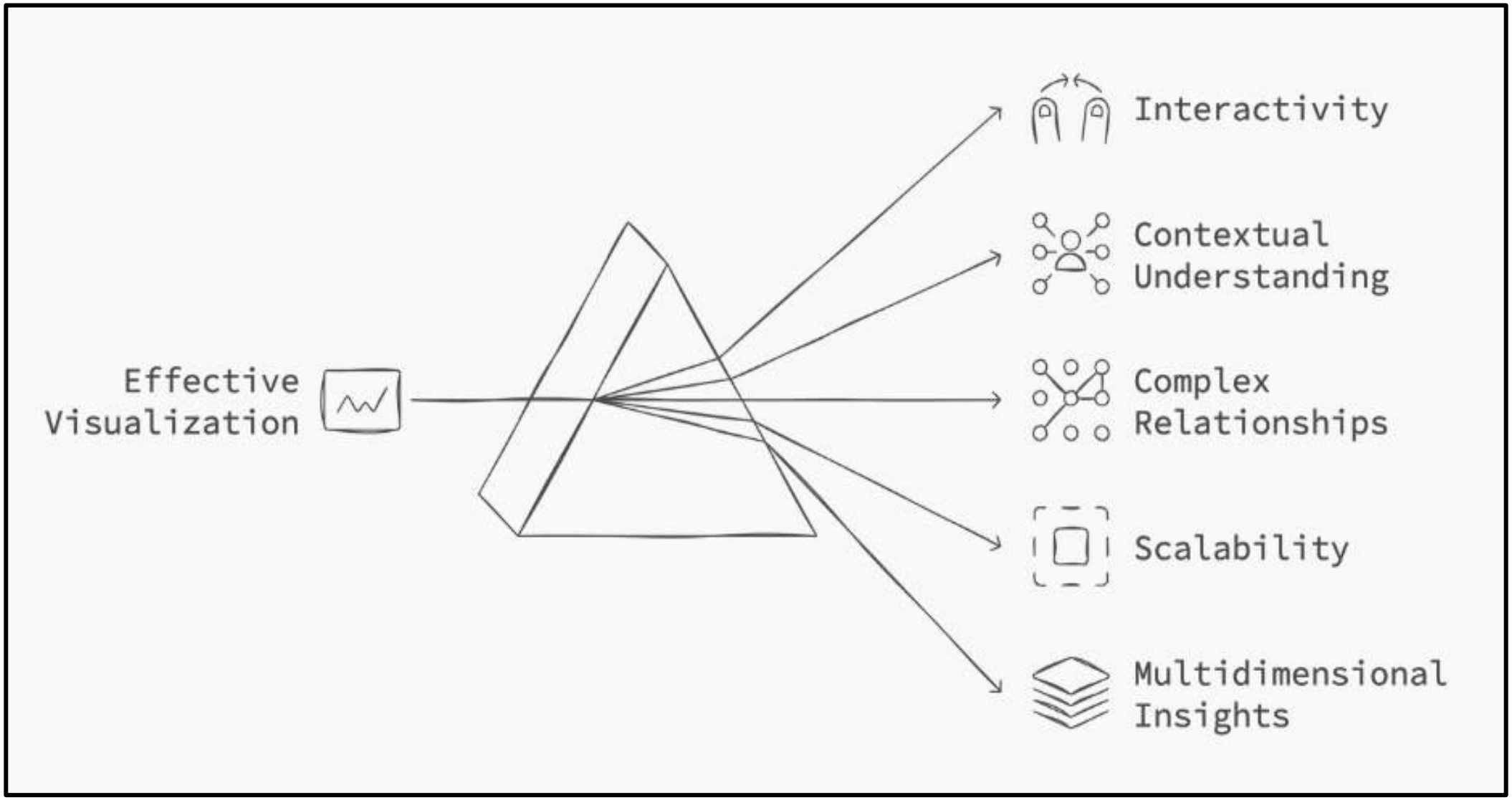

Figure: Knowledge Graph Visualization: Uses, Challenges and Benefits

https://www.falkordb.com/blog/knowledge-graph-visualization-insights/

Context Graphs

Definition: Dynamic representation of situational or conversational context around Knowledge.

Context: Captures when, why, and under what conditions relationships matter.

Example: Chatbots adapting answers based on user's intent, tone, or history.

Strength:

- Flexible, adaptive to scenarios.
- Adds nuance to knowledge graphs.
- Enables personalization and situational awareness.

Limitations:

- More complex to design and maintain.
- Requires continuous updates and contextual signals (time, location, user state).

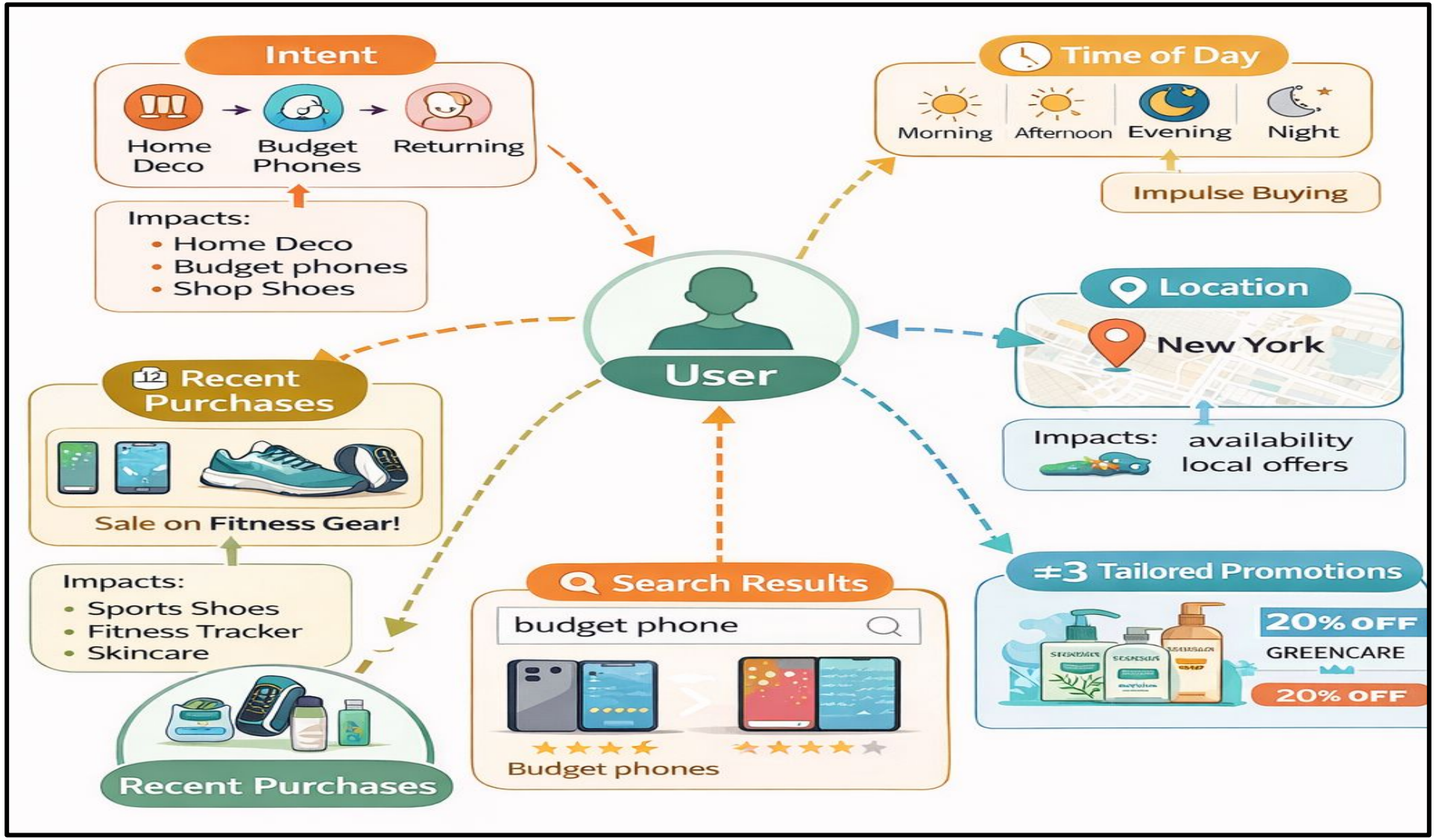

Figure: Context Graph For Dynamic User Behaviour Analysis in E-Commerce

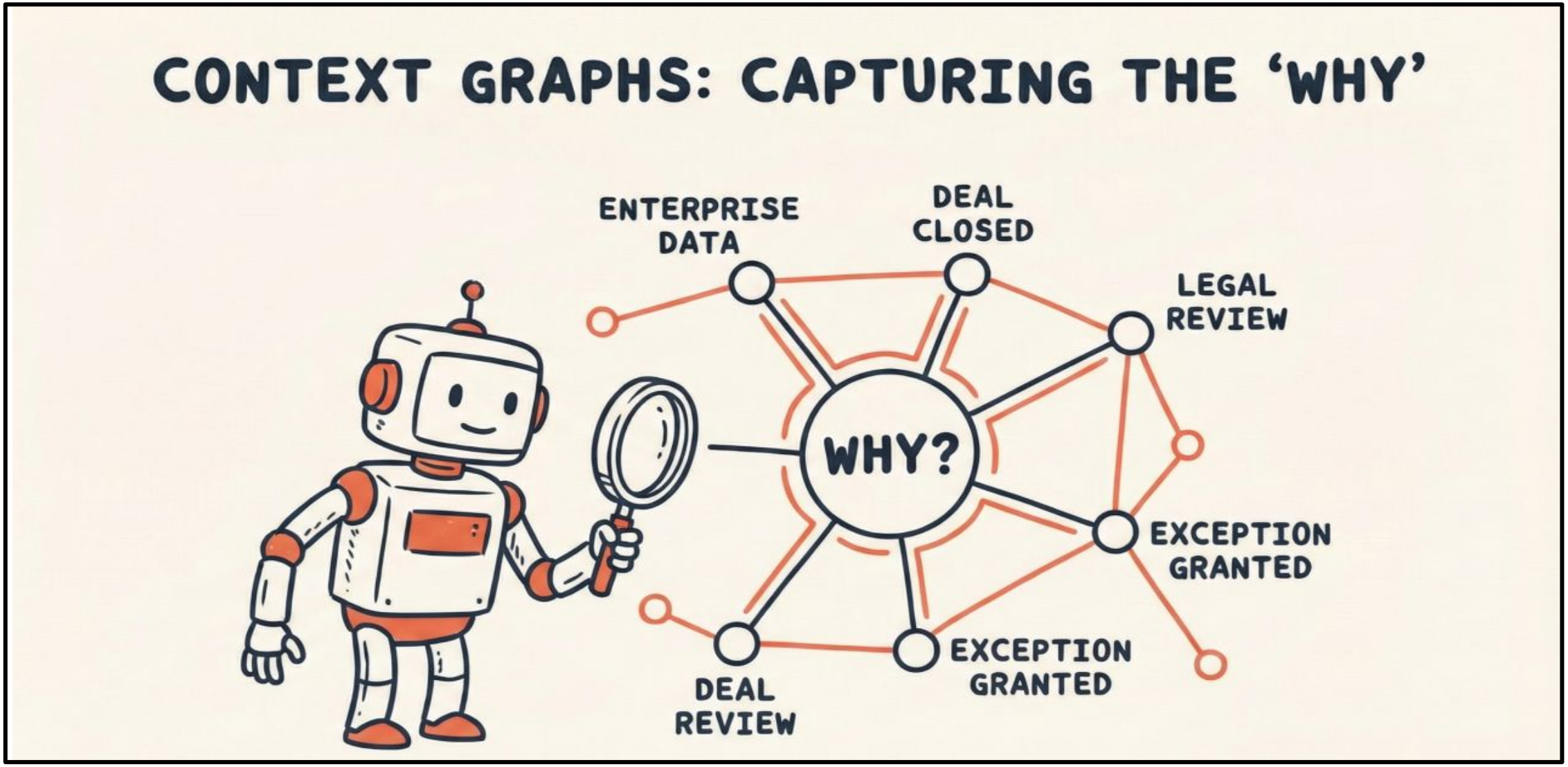

Figure: Context Graphs: Capturing The "Why" In The Age of AI

https://www.linkedin.com/pulse/context-graphs-capturing-why-age-ai-dharmesh-shah-oyyze

Ease vs Complexity

Aspect	Knowledge Graph	Context Graph
Ease of Use	Easier to build, query, and visualize	Harder - requires dynamic inputs and contextual modeling
Complexity	Lower - mostly static relationships	Higher - dynamic, multi-layered context
Best Use Case	Structured data, factual reasoning	Adaptive AI, personalization, scenario-driven design
Challenge	Ambiguity in meaning	Managing evolving context signals

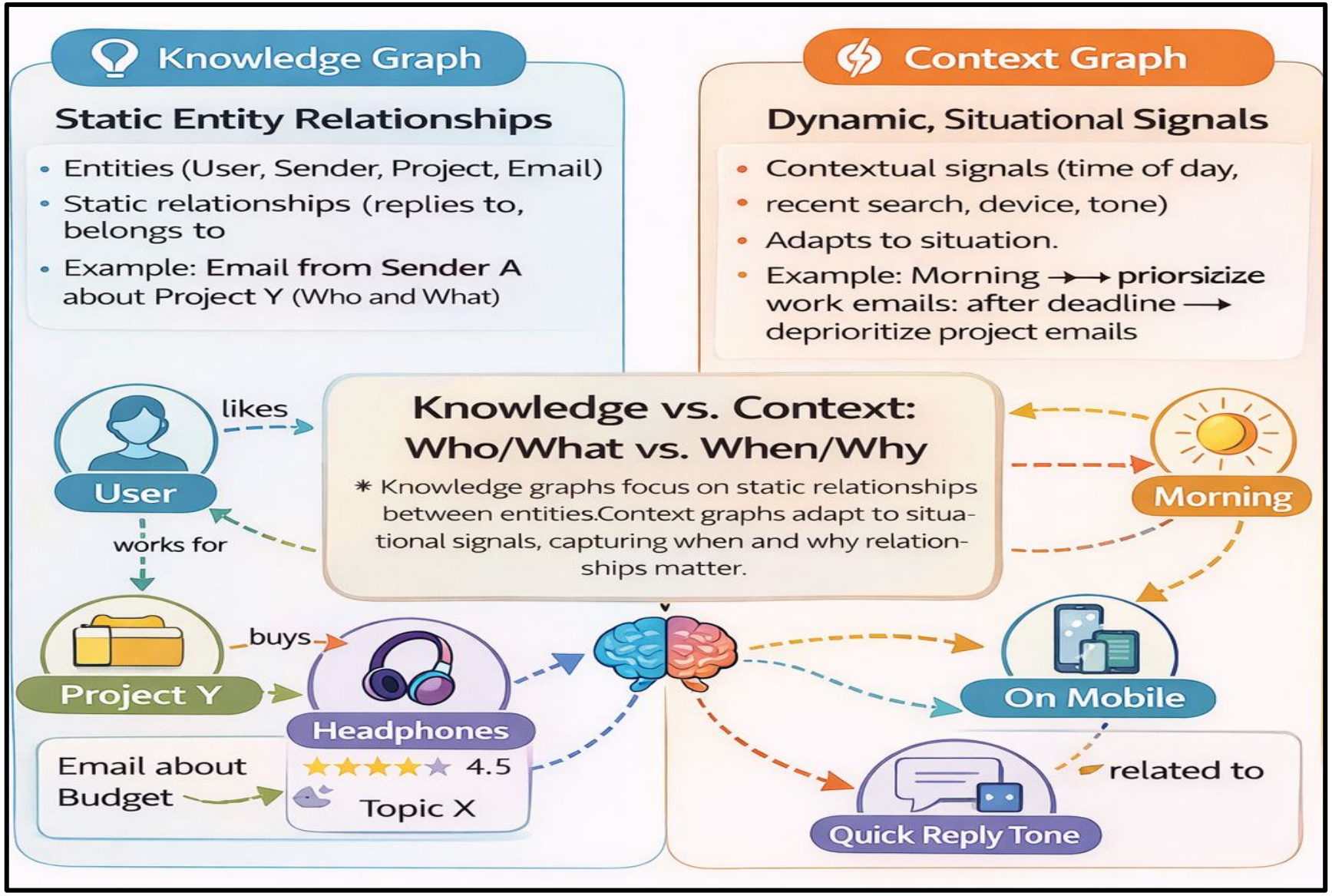

Figure: Understanding knowledge vs. context graphs

Role in AI Product Design

Knowledge Graphs:

Provide the foundation - the “map” of entities and relationships.

Context Graphs:

Provide the lens - adjusting the map to the situation at hand.

Together:

They enable AI systems that are both accurate (knowledge) and relevant (context).

Tools, Libraries, and Companies

Knowledge Graph:

Neo4j: Graph database for building/querying large knowledge graphs.

GraphDB (Ontotext): Enterprise semantic graph database with ontology support.

RDFLib (Python): Encodes knowledge in RDF triples; good for academic projects.

Google Knowledge Graph API: Provides entity and relationship data for apps.

NetworkX (Python): Lightweight graph analysis; ideal for prototypes.

Microsoft Graph : Connects organizational data (users, files, events) in graph form.

Context Graph:

TrustGraph

TrustGraph (Open Source): Context graph engine for dynamic signals (lineage, governance, time).

atlan

Atlan: Enterprise platform using context graphs for data governance and trust.

Google Cloud Microsoft Azure

Google AI Azure AI

Cloud AI Platforms (Google, Microsoft): Integrate context layers into APIs for personalization and situational awareness.

How Students Can Use Them in Brainstorming

1. **Start with Knowledge Graphs:** • Map entities and relationships relevant to the product idea.
   • Example: In a health app, connect symptoms → conditions → treatments.

2. **Layer Context Graphs:** • Add situational signals: time of day, user history, location, intent.
   • Example: The same health app adapts advice based on age, lifestyle, or recent activity.

3. **Prototype with Lightweight Tools:** • Use NetworkX or RDFLib for quick sketches.
   • Move to Neo4j or TrustGraph for realistic prototypes.

4. **Brainstorm Scenarios**• Ask: How does the product behave differently if the context changes?
   • Example: A travel AI recommends different destinations in summer vs winter, or for solo vs family trips.

Summary:

Think of Knowledge Graphs as the skeleton (structure) and Context Graphs as the muscles and nerves (adaptation). One gives stability, the other gives responsiveness.

Scenario:

Email Prioritization by Behavior

1. Knowledge Graph Approach

What it Captures: → Entities and relationships in the email ecosystem.

Structures: → Entities: User, Sender, Email, Topic, Attachment, Time.

Relationship: → User → frequently replies to → Sender A

Email → contains → Topic X
Sender B → belongs to → Project Y

Usage: →

- The system can infer that emails from Sender A or about Project Y are more important.
- Prioritization is based on static relationships (frequency, project membership, topic relevance).

Analogy: → A map of who's connected to whom and what topics matter.

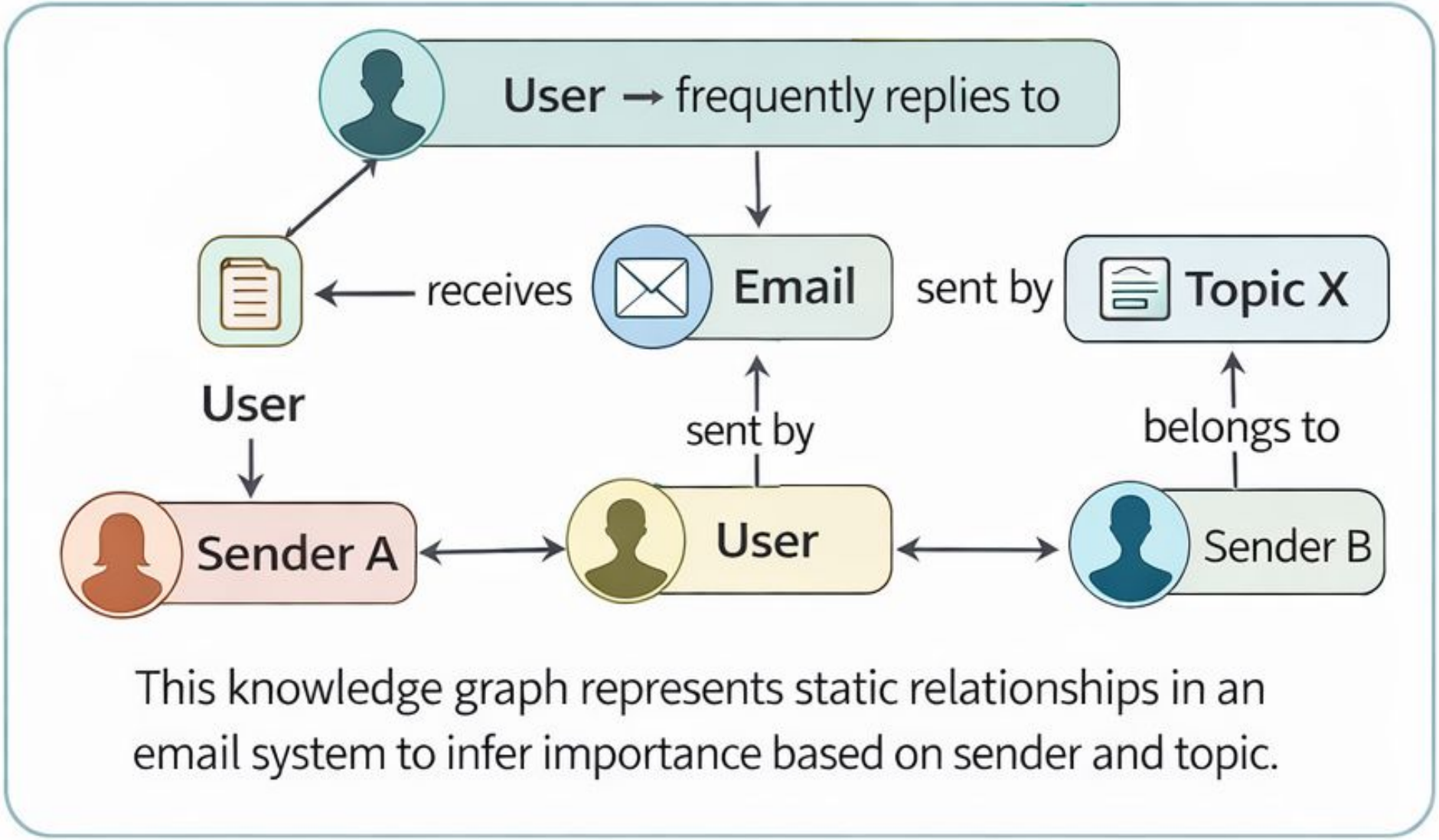

Figure: Knowledge Graph Examples

1. Context Graph Approach

What it Captures: → Situational and behavioral signals around those relationships.

Structures: → Contextual Signals: Time of day: User tends to reply to work emails in the morning.

Device: → User ignores long attachments when on mobile.

Recent Activity: → User ignores long attachments when on mobile.

Emotional Tone: → User ignores long attachments when on mobile.

Usage: →

- The system adapts prioritization dynamically
- Morning → prioritize work emails.
- Evening → prioritize personal/family emails.
- After a project deadline → deprioritize project-related threads.

Analogy: → A lens that changes the map depending on the situation.

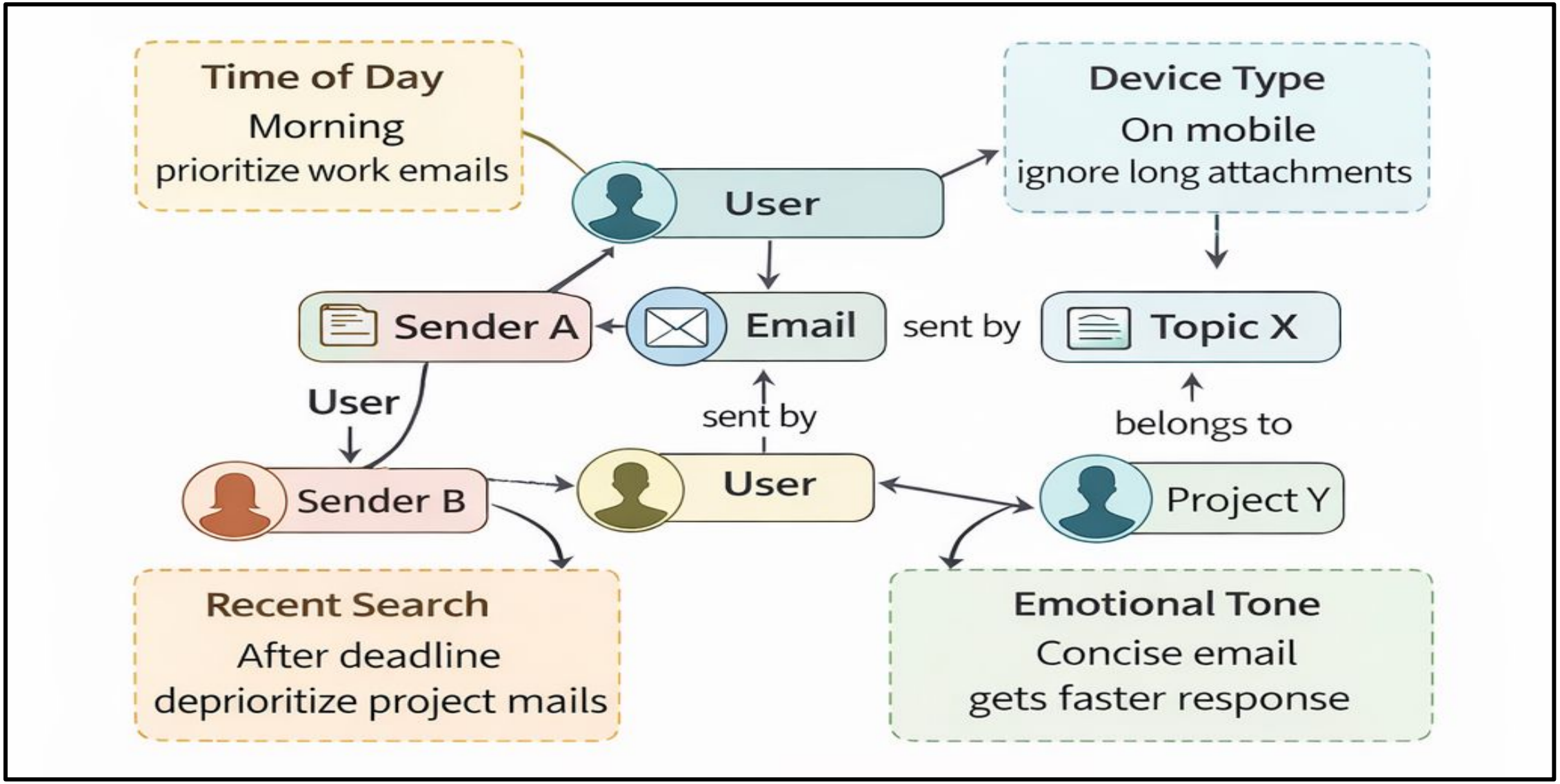

Figure: Context Graph Examples

Comparison Table

Aspect	Knowledge Graph	Context Graph
Focus	Entities + relationships	Situational signals + behavior
Nature	Static, factual	Dynamic, adaptive
Example Rule	“Sender A is important”	“Sender A is important only during project deadlines”
Strength	Clear structure	Personalization & adaptability
Limitations	Misses nuance	Complex to design & maintain

Classroom Exercise

STEP 1. → Students sketch a knowledge graph of email relationships (user ↔ sender ↔ topic).

STEP 2. → Extend it into a context graph by adding behavioral signals (time, device, recent searches).

STEP 3. → Discuss how prioritization changes when context shifts (e.g., morning vs evening, project vs vacation).

This scenario makes the distinction vivid:

- Knowledge Graph = “Who/What matters.”
- Context Graph = “When/Why it matters.